

📌 核心知识

- 树的表示法：first-child-next-sibling
- 二叉树
 - 性质
 - 树的遍历：前、中、后、层
 - 线索二叉树
 - 应用：文件系统、表达式树
- **二叉搜索树**
 - 各种操作，重点是插入和删除

1. 基本知识

1.1. 定义：它是一组节点，可以为空，如果不为空：

- 包含 1 个**根节点 (root)** r
- 有 0 个或多个**子树 (subtrees)** $T_1, \dots, T_k$ ，每个子树的根节点都和 r 通过一条**边 (edge)** 连接

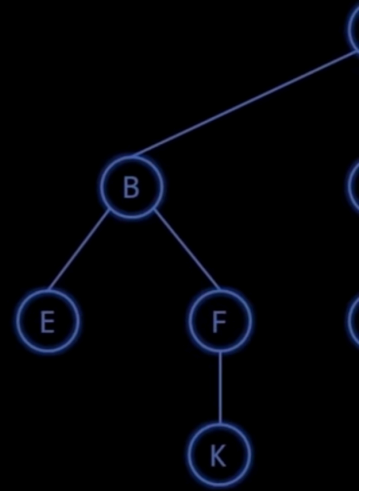
📌 注：

- 子树之间不会相互连接，因此每个节点都是某个子树的根节点
- 对于一棵有 N 个节点的树，它有 $N-1$ 条边

1.2. 基本术语

树的基本术语

1. 树的高度 = 树的深度 = 树的层数
2. 结点的深度: 它所在的层次
3. 结点的高度: 以它为根的子树的层数
4. 结点的度: 孩子的个数
5. 叶结点: 度为0的结点
6. 分支结点: 度大于0的结点
7. 树的度: 树中结点的最大度数
8. 有序树和无序树: 各结点的子树从左到右有次序, 不能互换, 称为有序树, 否则为无序树。
9. 路径和路径长度: 两个结点路上经过的结点序列(从上往下), 经过的边数就是路径长度
10. 森林: $m(m \geq 0)$ 棵互不相交的树的集合



树的基本术语

度为m的树和m叉树

度为m的树

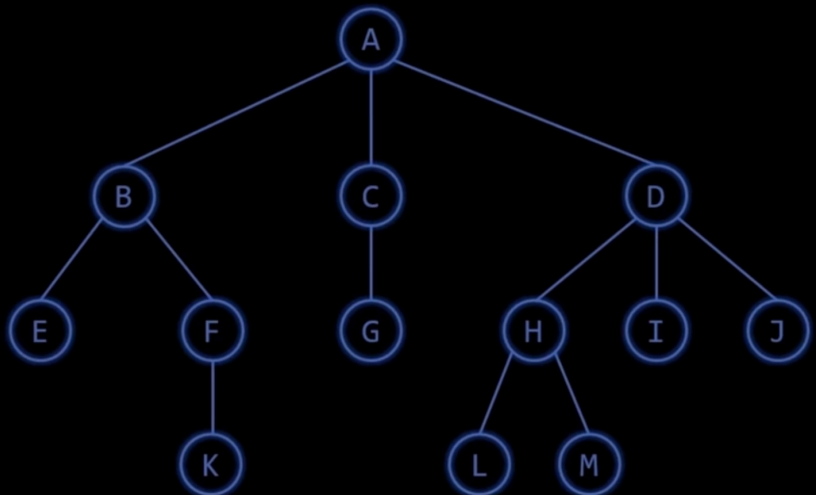
m叉树

相同点: 树里面结点的度最多为m

不同点:

度为m的树至少要有有一个结点的度等于m

m叉树所有结点的度可以都小于m, 甚至是空树



1.3.性质

树的性质

1. 总结点数 $n = \text{总边数} + 1 = \text{所有结点度数之和} + 1$
2. m 叉树第 i 层最多有 m^{i-1} 个结点 (度为 m 的树也是)
3. 高度为 h 的 m 叉树最多有 $\frac{m^h - 1}{m - 1}$ 个结点 (度为 m 的树也是)
4. 高度为 h 的 m 叉树最少有 h 个结点
度为 m 、高度为 h 的树最少有 $h - 1 + m$ 个结点
5. 具有 n 个结点的 m 叉树的最小高度为 $\lceil \log_m(n(m-1) + 1) \rceil$ (度为 m 的树也是)
6. 具有 n 个结点的 m 叉树的最大高度为 n
度为 m ，具有 n 个结点的树的最大高度为 $n - m + 1$

树的性质

5. 具有 n 个结点的 m 叉树的最小高度为 $\lceil \log_m(n(m-1) + 1) \rceil$ (度为 m 的树也是)

$$\frac{m^{h-1} - 1}{m - 1} + 1 \leq n \leq \frac{m^h - 1}{m - 1}$$

为了方便后续计算，等价地改为： $\frac{m^{h-1} - 1}{m - 1} < n \leq \frac{m^h - 1}{m - 1}$

同乘以 $m - 1$ 后加 1： $m^{h-1} < n(m - 1) + 1 \leq m^h$

取以 m 为底的对数： $h - 1 < \log_m(n(m - 1) + 1) \leq h$

等价于： $\log_m(n(m - 1) + 1) \leq h < \log_m(n(m - 1) + 1) + 1$

因为 h 必须是整数，可得： $h = \lceil \log_m(n(m - 1) + 1) \rceil$

个人心得：

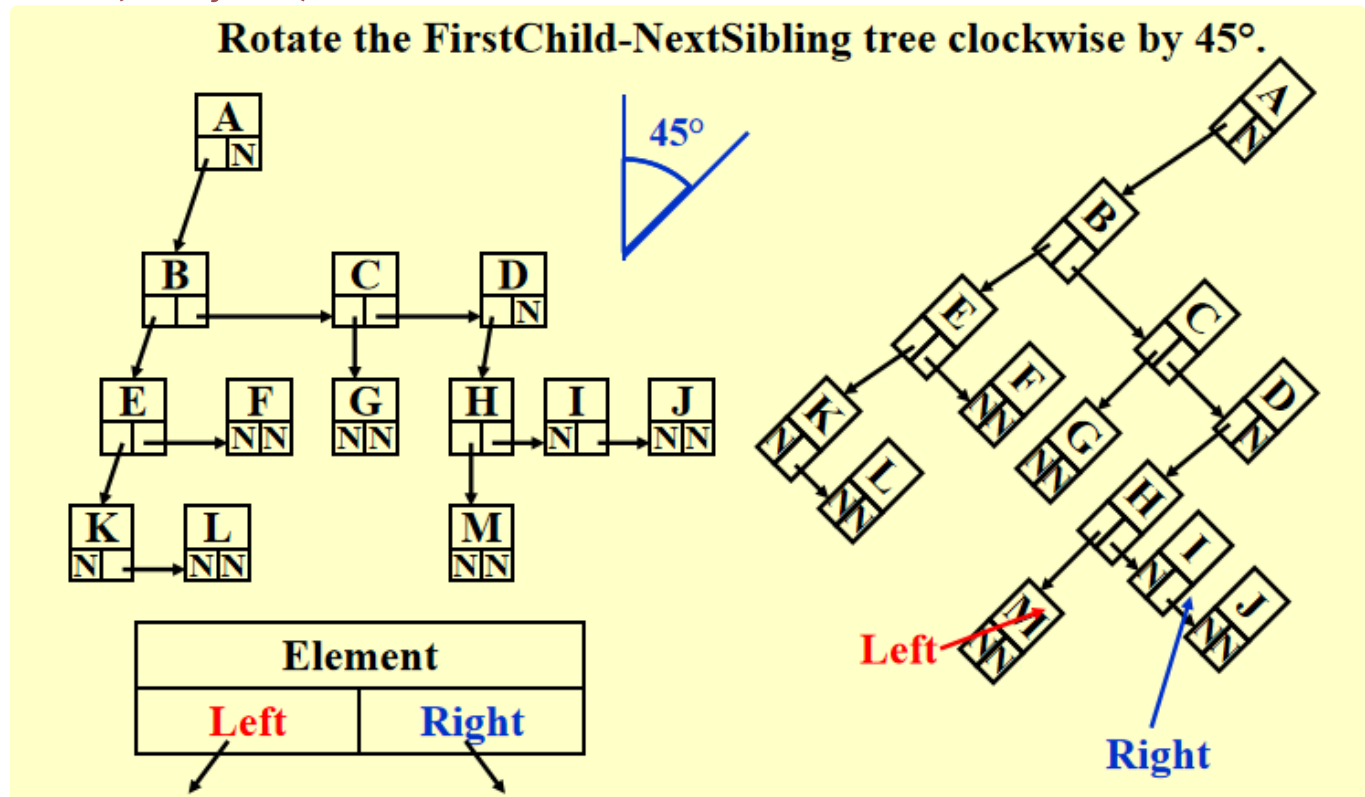
- 对于概念搞明白深度、高度两个概念。深度是从根节点开始算的层数（根节点的深度为 0）；高度是从叶节点开始算的（叶节点的深度为 0）（那两个公式算是特例）

- 区分好度为m的树和m叉树（度为m说明至少有一个节点的度为m；m叉树是一开始在构建树的时候设计好了度为m，但经过删除插入等操作之后它的度不一定是m，甚至可以为空树）
- 任何一棵树都可以转化成二叉树
- 树都可以用一个一维数组来表示（每个位置可以存储当前节点的父节点的index）

2.Binary Tree 二叉树

2.1.基本概念

二叉树 (binary tree): 每个节点拥有不超过两个孩子的树（最大度为2的树）



补充：一般树（左图）的后序遍历 = 由上述方法得到的二叉树（右图）的中序遍历

2.2.Tree Traversals

树的遍历 (tree traversals): 对树的每个节点都访问一次，时间复杂度为 $O(N)$

2.2.1. 前序遍历 (preorder traversal)

```
void preorder(tree_ptr tree)
{
    if (tree)
    {
        visit(tree); // visit=printf
        for (each child C of tree)
            preorder(C);
    }
}
```

```
        preorder(C);
    }
}
```

2.2.2. 后序遍历 (postorder traversal)

```
void postorder(tree_ptr tree)
{
    if (tree)
    {
        for (each child C of tree)
            postorder(C);
        visit(tree);
    }
}
```

2.2.3. 层序遍历 (levelorder traversal)

```
void levelorder(tree_ptr tree)
{
    enqueue(tree);
    while (queue is not empty)
    {
        visit(T = dequeue());
        for (each child C of T)
            enqueue(C);
    }
}
```

2.2.4. 中序遍历 (inorder traversal)

```
// Recursion
void inorder(tree_ptr tree)
{
    if (tree)
    {
        inorder(tree->Left);
        visit(tree->Element);
        inorder(tree->Right);
    }
}

// Iteration(using stack)
```

```

void iter_inorder(tree_ptr tree)
{
    Stack S = CreateStack(MAX_SIZE);
    for (;;)
    {
        for (; tree; tree = tree->left)
            Push(tree, S);
        tree = Top(S);
        Pop(S);
        if (!tree)
            break;
        visit(tree->Element);
        tree = tree->Right;
    }
}

```

✎ 总结点：

- 知道前序或者后序遍历 + 中序遍历，可以确定唯一的一棵树
- 知道前序遍历 + 后序遍历，一般情况下无法确定树的形状

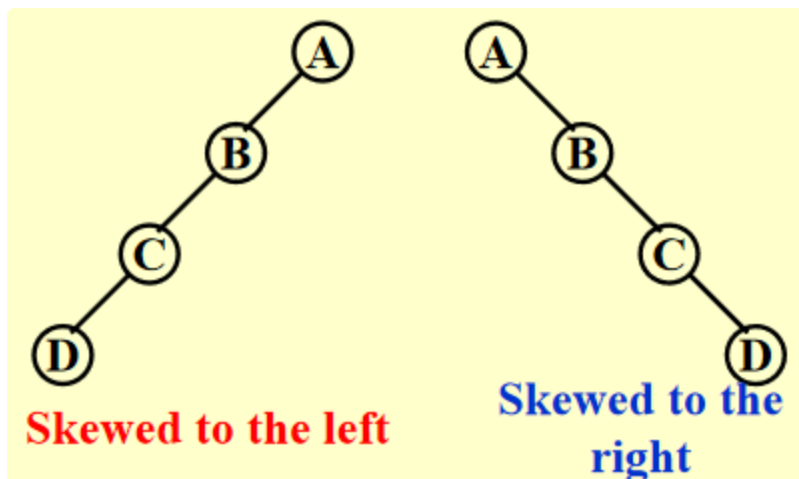
2.3. Threaded Tree 线索二叉树

线索二叉树 (threaded binary trees)很好地利用了闲置的节点，具体规则如下：

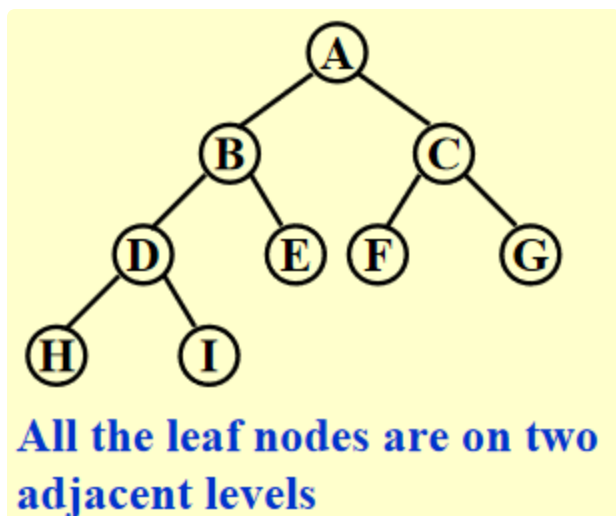
- 如果 `Tree->Left` 为空，将它指向**中序遍历中的前一个节点**
- 如果 `Tree->Right` 为空，将它指向**中序遍历中的后一个节点**
- 有一个**头节点**(dummy node)，使得**最左边**和**最右边**孩子分别指向这个节点的左右孩子

2.4 Special tree

2.4.1 skewed tree 斜二叉树



2.4.2 complete binary tree 完全二叉树



2.5 性质

- 第 i 层 ($i \geq 1$) 最多拥有 $2^i - 1$ 个节点；深度为 k 的二叉树最多有 $2^{k+1} - 1$ 个节点 ($k \geq 1$)
- 对于非空的二叉树， $n_0 = n_2 + 1$ ，其中 n_0 是叶子节点个数， n_2 是度为 2 的节点个数

推论：

- 推论1：对于一棵完全二叉树，易知 $n_1 = 0$ 或 1 。由 $n_0 = n_2 + 1$ 知：
 - 完全二叉树有奇数个节点时， $n_1 = 0$
 - 完全二叉树有偶数个节点时， $n_1 = 1$
- 推论2： n 叉树的叶子节点个数与 n_1 无关，且 $n_0 = \sum_{i=2}^n (i-1)n_i + 1$

3.Binary Search Tree

3.1.Definition

- A binary search tree is a binary tree. It may be empty. If it is not empty, it satisfies the following properties:
 - (1) Every node has a key which is an integer, and the keys are distinct.
 - (2) The keys in a nonempty left subtree must be smaller than the key in the root of the subtree.
 - (3) The keys in a nonempty right subtree must be larger than the key in the root of the subtree.
 - (4) The left and right subtrees are also binary search trees.

3.2.代码实现

3.2.1 Find

- 递归版本

```
Position Find( ElementType X, SearchTree T )
{
    if ( T == NULL )
        return NULL; /* not found in an empty tree */
    if ( X < T->Element ) /* if smaller than root */
        return Find( X, T->Left ); /* search left subtree */
    else
        if ( X > T->Element ) /* if larger than root */
            return Find( X, T->Right ); /* search right subtree */
        else /* if X == root */
            return T; /* found */
}

 $T(N) = S(N) = O(d)$  where  $d$  is the depth of X
```

- 迭代版本

```
Position Iter_Find( ElementType X, SearchTree T )
{
    /* iterative version of Find */
    while ( T ) {
        if ( X == T->Element )
            return T; /* found */
        if ( X < T->Element )
            T = T->Left; /* move down along left path */
        else
            T = T->Right; /* move down along right path */
    } /* end while-loop */
    return NULL; /* not found */
}
```

- FindMin

```
Position FindMin( SearchTree T )            $T(N) = O(d)$ 
{
    if ( T == NULL )
        return NULL; /* not found in an empty tree */
    else
        if ( T->Left == NULL ) return T; /* found left most */
        else return FindMin( T->Left ); /* keep moving to left */
}
```

- FindMax

```
Position FindMax( SearchTree T )            $T(N) = O(d)$ 
{
    if ( T != NULL )
        while ( T->Right != NULL )
            T = T->Right; /* keep moving to find right most */
    return T; /* return NULL or the right most */
}
```

3.2.2 插入

```

SearchTree Insert( ElementType X, SearchTree T )
{
    if ( T == NULL ) { /* Create and return a one-node tree */
        T = malloc( sizeof( struct TreeNode ) );
        if ( T == NULL )
            FatalError( "Out of space!!!" );
        else {
            T->Element = X;
            T->Left = T->Right = NULL; }
    } /* End creating a one-node tree */
    else /* If there is a tree */
        if ( X < T->Element )
            T->Left = Insert( X, T->Left );
        else
            if ( X > T->Element )
                T->Right = Insert( X, T->Right );
            /* Else X is in the tree already; we'll do nothing */
    return T; /* Do not forget this line!! */
}

```

$T(N) = O(d)$

3.2.3.删除

```

SearchTree Delete( ElementType X, SearchTree T )
{
    Position TmpCell;
    if ( T == NULL ) Error( "Element not found" );
    else if ( X < T->Element ) /* Go left */
        T->Left = Delete( X, T->Left );
    else if ( X > T->Element ) /* Go right */
        T->Right = Delete( X, T->Right );
    else /* Found element to be deleted */
        if ( T->Left && T->Right ) { /* Two children */
            /* Replace with smallest in right subtree */
            TmpCell = FindMin( T->Right );
            T->Element = TmpCell->Element;
            T->Right = Delete( T->Element, T->Right ); } /* End if */
        else { /* One or zero child */
            TmpCell = T;
            if ( T->Left == NULL ) /* Also handles 0 child */
                T = T->Right;
            else if ( T->Right == NULL ) T = T->Left;
            free( TmpCell ); } /* End else 1 or 0 child */
    return T;
}

```

$T(N) = O(h)$ where h is the height of the tree

4.一般情况分析

- **Question:** Place n elements in a binary search tree. How high can this tree be?
- **Answer:** The height depends on *the order of insertion*.

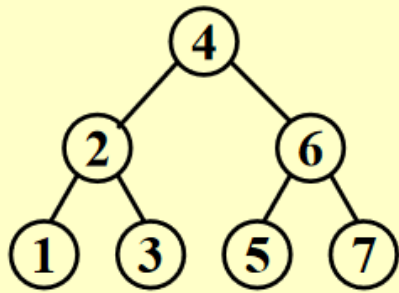
Example

[[Example]] Given elements 1, 2, 3, 4, 5, 6, 7. Insert them into a binary search tree in the orders:

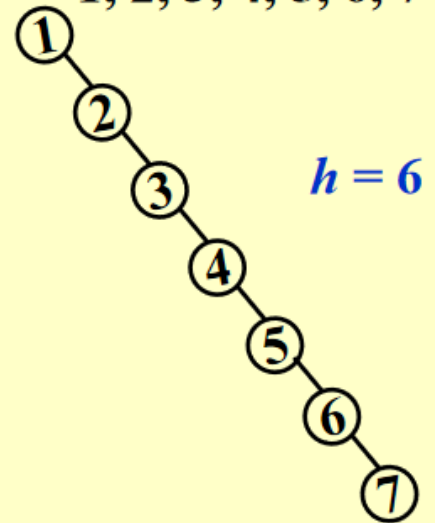
4, 2, 1, 3, 6, 5, 7

and

1, 2, 3, 4, 5, 6, 7



$h = 2$



$h = 6$